

Component Attributes & Props

In this lesson, we are going to look at component attributes and props. We will create a `NavLink` component that accepts attributes and props. The difference between attributes and props is that attributes are for actual defined HTML attributes, while props are for custom attributes.

We have our navigation links styled and working as expected. However, we can make our navigation links more dynamic by creating a `NavLink` component that accepts attributes.

Run the following command to create a new component:

```
php artisan make:component NavLink
```

This creates a new component in the `View/Components` directory. Open the `nav-link.blade.php` file and replace the content with the following for now:

```
<a>
    {{ $slot }}
</a>
```

The `$slot` variable is a special variable that contains the content passed to the component. In this case, it will contain the text of the link.

So in the `header.blade.php` file, replace the home link with the following:

```
<x-nav-link href="{{ url('/') }}">Home</x-nav-link>
```

You should see the text "Home" displayed on the page. However, the link is not working yet. Let's update the `NavLink` component to include the `href` attribute.

Attributes

We can use the `$attributes` variable to pass all attributes to the component. This allows us to pass any attribute to the component without explicitly defining them.

Open the `nav-link.blade.php` file and update it as follows:

```
@php
    echo $attributes;
@endphp

<a>
```

```
    {{$slot}}  
</a>
```

You should see the `href` attribute printed on the page and the class attribute.

Get rid of the `@php` block. I just wanted to show you how to use it.

Now simply add the `@attributes` variable to the `a` tag:

```
<a {{ $attributes }}>  
    {{ $slot }}  
</a>
```

This is different from `props`. We will look at props soon, but attributes are for actual defined HTML attributes, while props are for custom attributes. It should now work as expected.

Now one of my reasons for using a nav link component is so we don't have to pass so many attributes to the `a` tag. But now we are passing the same number of attributes to the `x-nav-link` component. We can move the class attribute to the `NavLink` component.

Open the `nav-link.blade.php` file and update it as follows:

```
<a {{$attributes}} class="text-white hover:underline py-2 {{ request()->is('/') ?  
'text-yellow-400 font-bold' : '' }}">  
    {{$slot}}  
</a>
```

However, now, it always checks for the homepage.

Props

We can pass custom attributes to the component using props. Let's create a prop called `active` that will determine if the link is active.

We need to define any custom props using the `@props` directive at the top of the component file. Open the `nav-link.blade.php` file and update it as follows:

```
@props(['active'])  
  
<a {{$attributes}} class="text-white hover:underline py-2 {{ request()->is('/') ?  
'text-yellow-400 font-bold' : '' }}">  
    {{$slot}}  
</a>
```

We are defining a prop called `active` that we can use in the component. By doing this, if we pass in the `active` prop, it will not be included in the `$attributes` variable.

Default Values

We can also give it a default value. Let's set the default value of `active` to `false`. Update the `@props` directive as follows:

```
@props(['active' => false])
```

Now, let's check for the `active` prop and add the `font-bold` and `text-yellow-400` classes if it is true. Update the `a` tag as follows:

```
<a {{$attributes}} class="text-white hover:underline py-2 {{ $active ? 'text-yellow-400 font-bold' : '' }}">
    {{$slot}}
</a>
```

Using Props

Now, in the `header.blade.php` file, we can pass the `active` prop to the `NavLink` component. Update the home link as follows:

```
<x-nav-link href="{{ url('/') }}" active="false">Home</x-nav-link>
```

Notice that the link is active now even though we passed `false`. This is because the `"false"` is being passed as a string, which will evaluate to `true`. To fix this, we need to pass a boolean value. Update the `active` prop as follows:

```
<x-nav-link href="{{ url('/') }}" :active="false">Home</x-nav-link>
```

We just added the `:` before the prop to pass it as a boolean. Now the link should not be active. If you change it to `true`, the link will be active. The `:` is not just for boolean values; it is for any dynamic value.

Now instead of passing true or false, pass the result of the `request()->is('/')` function. Update the `active` prop as follows:

```
<x-nav-link href="{{ url('/') }}" :active="request()->is('/')">Home</x-nav-link>
```

URL Prop

You could leave it like this, but I actually want to pass in the url as a prop because we may want to use the url in other ways in the future and this will make it easier. I also just think it looks cleaner.

Update the `@props` directive as follows:

```
@props(['url' => '/', 'active' => false])
```

Now edit the `a` tag to use the `url` prop:

```
<a href="{{ $url }}" class="text-white hover:underline py-2 {{ $active ? 'text-yellow-400 font-bold' : '' }}">
    {{$slot}}
</a>
```

Now we can simply change all of our `<nav-link>` components to use the `url` prop. Update the links in the `header.blade.php` file as follows:

```
<x-nav-link url="/" :active="request()->is('/')">Home</x-nav-link>
<x-nav-link url="/jobs" :active="request()->is('jobs')">All Jobs</x-nav-link>
<x-nav-link url="/jobs/saved" :active="request()->is('jobs/saved')">Saved Jobs</x-nav-link>
<x-nav-link url="/login" :active="request()->is('login')">Login</x-nav-link>
<x-nav-link url="/register" :active="request()->is('register')">Register</x-nav-link>
<x-nav-link url="/profile" :active="request()->is('profile')"><i class="fa fa-user mr-1"></i> Profile
</x-nav-link>
```

Let's leave the create job link along because we are going to have a separate component for button links.

Icon Prop

Open the `views/layout.blade.php` file and let's cut the `<i class='fa fa-gauge'></i>`. Now paste it in the `views/components/nav-link.blade.php` file right before the `{{ $slot }}`:

```
<a
    href="{{ $url }}"
    class="text-white hover:underline py-2 {{ $active ? 'text-yellow-400 font-bold' : '' }}"
>
    <i class="fa fa-gauge mr-1"></i> {{$slot}}
</a>
```

Let's add a new prop at the top of the `views/components/nav-link.blade.php` file:

```
@props(['url' => '/', 'active' => false, 'icon' => null])
```

We set the default value of the `icon` prop to `null`. Now replace the `<i class='fa fa-gauge'></i>` with the following code:

```
<a
  href="{{ $url }}"
  class="text-white hover:underline py-2 {{ $active ? 'text-yellow-400 font-bold'
: '' }}"
>
  @if($icon)
    <i class="fa fa-{{ $icon }}" mr-1"></i>
  @endif {{ $slot }}
</a>
```

Now you can pass in an icon prop to the `nav-link` component. Let's test it out.

Open the `views/header.blade.php` file and add the gauge icon to the dashboard link:

```
<x-nav-link url="/dashboard" :active="request()->is('dashboard')" icon="gauge"
>Dashboard</x-nav-link
>
```

Now you should see the gauge icon next to the dashboard link.